main

October 30, 2025

[1]:
```
def plot_confusion_matrix(y,y_pred):
# Plot confusion matrix (raw and normalized) for the last cross-validated␣
 ↪predictions (y_pred)
    labels = ["Low", "Medium", "High"]
    cm = confusion_matrix(y, y_pred, labels=labels)

    # normalized by true-label (row) counts
    with np.errstate(all='ignore'):
        cm_norm = cm.astype(float) / cm.sum(axis=1)[:, None]
        cm_norm = np.nan_to_num(cm_norm)  # replace NaN from zero-rows if any

    fig, axes = plt.subplots(1, 2, figsize=(12, 5))

    # Raw counts
    ax = axes[0]
    im = ax.imshow(cm, interpolation='nearest', cmap='Blues')
    ax.set_title(f"Confusion Matrix (counts) - {name if 'name' in globals()␣
 ↪else ''}")
    ax.set_xticks(range(len(labels))); ax.set_yticks(range(len(labels)))
    ax.set_xticklabels(labels); ax.set_yticklabels(labels)
    ax.set_ylabel('True label'); ax.set_xlabel('Predicted label')
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            ax.text(j, i, f"{cm[i, j]}", ha="center", va="center",␣
 ↪color="black")
    fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)

    # Normalized
    ax = axes[1]
    im2 = ax.imshow(cm_norm, interpolation='nearest', cmap='Blues', vmin=0,␣
 ↪vmax=1)
    ax.set_title("Confusion Matrix (normalized by true class)")
    ax.set_xticks(range(len(labels))); ax.set_yticks(range(len(labels)))
    ax.set_xticklabels(labels); ax.set_yticklabels(labels)
    ax.set_ylabel('True label'); ax.set_xlabel('Predicted label')
    for i in range(cm_norm.shape[0]):
        for j in range(cm_norm.shape[1]):
```

```
            ax.text(j, i, f"{cm_norm[i, j]:.2f}", ha="center", va="center",␣
 ↪color="black")
    fig.colorbar(im2, ax=ax, fraction=0.046, pad=0.04)

    plt.tight_layout()
    plt.show()
```

1 Preprocessing

[2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_squared_error
from sklearn.ensemble import RandomForestRegressor
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import roc_curve, auc
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_predict,␣
 ↪GridSearchCV
from sklearn.metrics import classification_report, confusion_matrix,␣
 ↪accuracy_score
from sklearn.metrics import balanced_accuracy_score
# 1. Load and clean dataset
df = pd.read_excel("downsample_correct_columns.xlsx")

df = df.dropna(subset=[
    "Incident State",
    "Date Of Incident",
    "Quantity Released",
    "Unit Of Measure",
    "Hazardous Class",
    "Time Of Incident",
    "Mode Of Transportation",
    "Total Evacuation Hours",
    "Public Evacuated",
    "Major Artery Hours Closed",
    "Undeclared Hazmat Shipment Ind",
    "Total Amount Of Damages"
])

# Convert and extract temporal features
```

```
df["Date Of Incident"] = pd.to_datetime(df["Date Of Incident"])
df["Incident Month"] = df["Date Of Incident"].dt.month

# Remove non-numeric hazard classes
df = df[df["Hazardous Class"].apply(lambda v: not isinstance(v, str))]

def get_season(month):
    if month in [12, 1, 2]:
        return 'Winter'
    elif month in [3, 4, 5]:
        return 'Spring'
    elif month in [6, 7, 8]:
        return 'Summer'
    else:
        return 'Fall'

df['Season'] = df['Date Of Incident'].dt.month.apply(get_season)

def get_time_of_day(time):
    t = int(time)
    hour = t // 100
    if 5 <= hour < 12:
        return 'Morning'
    elif 12 <= hour < 14:
        return 'Noon'
    elif 14 <= hour < 18:
        return 'Afternoon'
    elif 18 <= hour < 22:
        return 'Evening'
    else:
        return 'Night'

df['Time Of Day'] = df['Time Of Incident'].apply(get_time_of_day)
df = df.drop(['Date Of Incident', 'Time Of Incident'], axis=1)

# 2. Encode and prepare data
X = pd.get_dummies(df.drop(columns=['Total Amount Of Damages']),␣
 ↪drop_first=True)
y = df['Total Amount Of Damages']
```

2 Remove outlier from y and use that mask to filter X

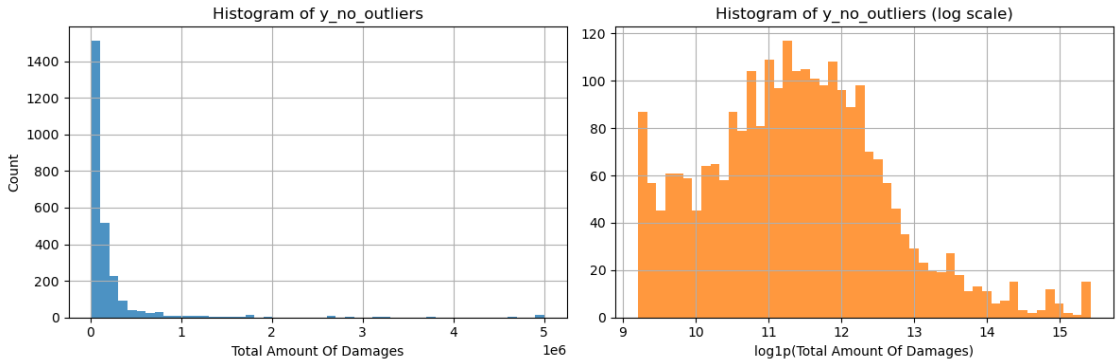
[3]:
```
# 3. Remove outliers
mean = y.mean()
std = y.std()
```

```
z_scores = (y - mean) / std
mask = abs(z_scores) < 3
X_no_outliers, y_no_outliers = X[mask], y[mask]

print(f"Removed {len(y) - len(y_no_outliers)} outliers out of {len(y)} samples.
  ↪")
```

```
Removed 21 outliers out of 2614 samples.
```

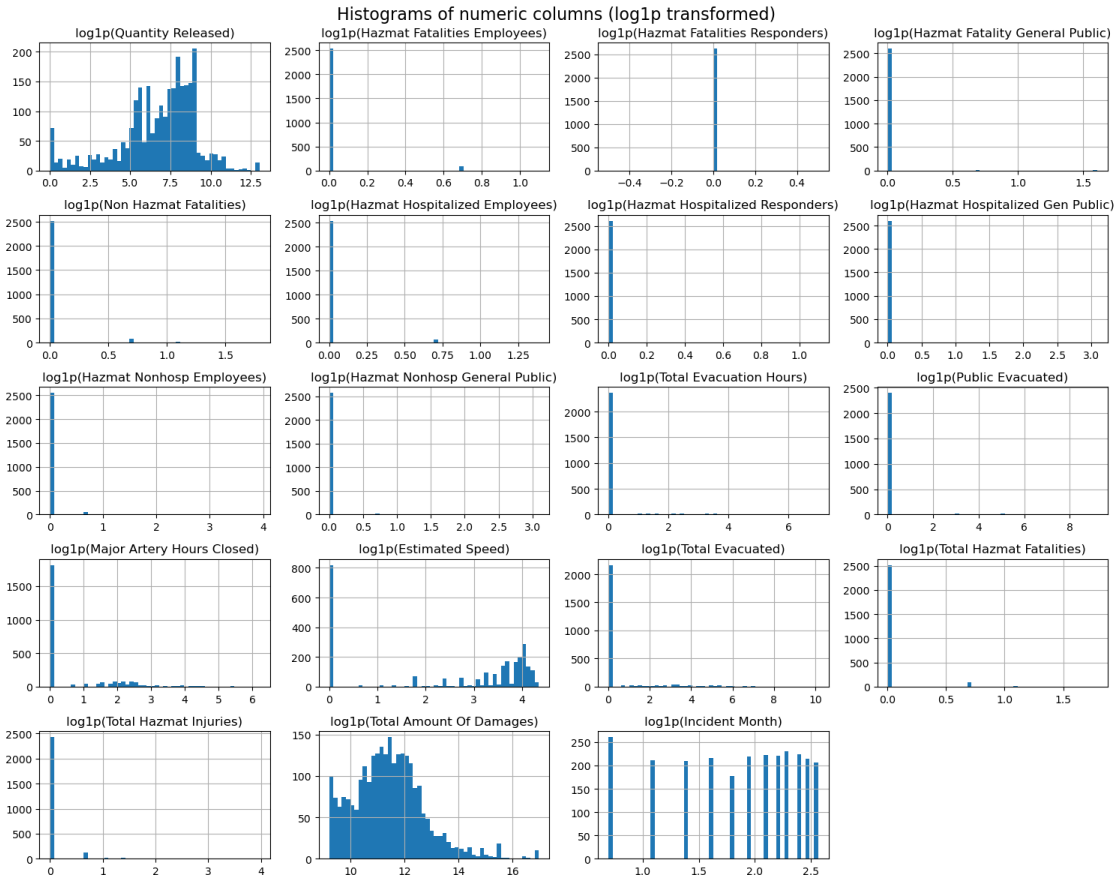
2.1 After remove outlier on y visualize distribution of remaining

[4]:
```
plt.figure(figsize=(12,4))

plt.subplot(1,2,1)
plt.hist(y_no_outliers, bins=50, color='C0', alpha=0.8)
plt.xlabel('Total Amount Of Damages')
plt.ylabel('Count')
plt.title('Histogram of y_no_outliers')
plt.grid(True)

plt.subplot(1,2,2)
plt.hist(np.log1p(y_no_outliers), bins=50, color='C1', alpha=0.8)
plt.xlabel('log1p(Total Amount Of Damages)')
plt.title('Histogram of y_no_outliers (log scale)')
plt.grid(True)

plt.tight_layout()
plt.show()
```



2.2 A lot of numeric columns are useless only keeping “quantity release” and “estimated speed” (after log transform)

[5]:
```
# Log-transform all numeric columns (use np.log1p to handle zeros) and plot histograms
# 'num' is already defined in the notebook as df.select_dtypes(include=[np.number])
num_log = np.log1p(df.select_dtypes(include=[np.number]))

# optional: rename columns to indicate transformation (keeps original columns intact)
num_log.columns = [f"log1p({c})" for c in num_log.columns]

# Plot histograms for all log-transformed numeric columns
num_log.hist(bins=50, figsize=(15, 12))
plt.suptitle("Histograms of numeric columns (log1p transformed)", fontsize=16)
plt.tight_layout()
plt.show()
```



[6]:
```
# Keep only "Quantity Released" among numeric columns in X (and X_no_outliers␣
 ↪if present)
num_cols = X_no_outliers.select_dtypes(include=[np.number]).columns.tolist()
cols_to_drop = [c for c in num_cols if c not in ("Quantity Released",␣
 ↪"Estimated Speed")]

# Drop numeric cols except Quantity Released and Estimated Speed
X_no_outliers = X_no_outliers.drop(columns=cols_to_drop)
```

2.3 Since there are only two columns numeric might as well change to clssification problem

3 Create a 3-class target from log(y_no_outliers):

4 -1 if z <= -1, 1 if z >= 1, 0 otherwise

[7]:
```
# Create a 3-class target from log(y_no_outliers):
# -1 if z <= -1,  1 if z >= 1,  0 otherwise
y_no_outliers_log = np.log1p(y_no_outliers)
mean_log = y_no_outliers_log.mean()
std_log = y_no_outliers_log.std()
z_log = (y_no_outliers_log - mean_log) / std_log

y_transform = z_log.apply(lambda v: "Low" if v <= -1 else ("High" if v >= 1␣
 ↪else "Medium"))
print("Class counts:\n", y_transform.value_counts())
```

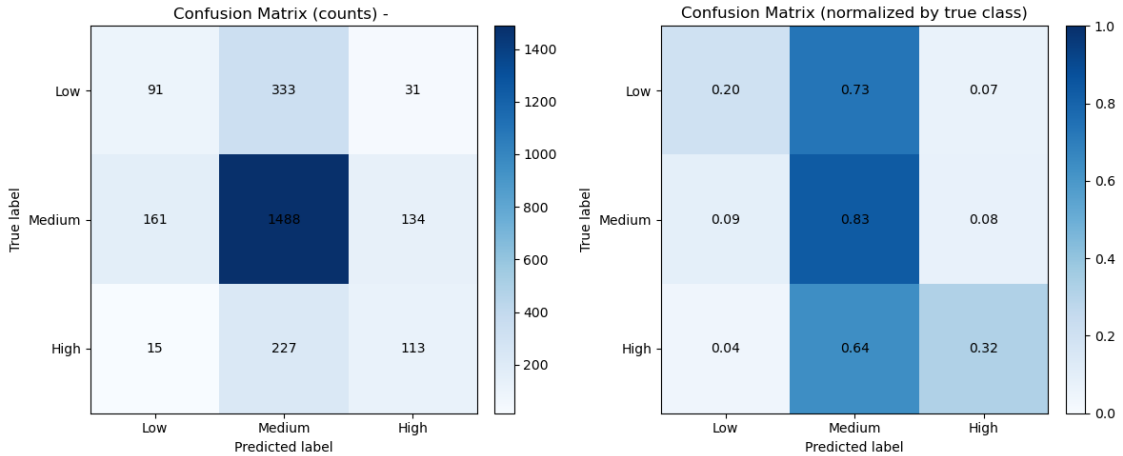
```
Class counts:
 Total Amount Of Damages
Medium    1783
Low        455
High       355
Name: count, dtype: int64
```

5 Base Model using KNN

[8]:
```
# Prepare features/labels (reuse existing variables)
X_knn = X_no_outliers.copy().astype(float)
y_knn = y_transform.loc[X_knn.index]



# Base pipeline (scaling + KNN)
knn_pipe = make_pipeline(StandardScaler(), KNeighborsClassifier())

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

```
# Hyperparameter tuning on n_neighbors using multiclass ROC-AUC (ovr)
param_grid = {
    "kneighborsclassifier__n_neighbors": [3, 5, 7, 9, 11, 15]
}
grid = GridSearchCV(knn_pipe, param_grid, cv=cv, scoring='balanced_accuracy',␣
 ↪n_jobs=-1, verbose=1)
grid.fit(X_knn, y_knn)
print("Best params (n_neighbors):", grid.best_params_)
best_knn = grid.best_estimator_

# cross-validated aggregated predictions (unbiased)
y_pred = cross_val_predict(best_knn, X_knn, y_knn, cv=cv, n_jobs=-1)

print(f"(5-fold CV aggregated predictions) Balanced Accuracy:",␣
 ↪balanced_accuracy_score(y_knn, y_pred))
print(classification_report(y_knn, y_pred, digits=4))
plot_confusion_matrix(y_knn, y_pred)
```

```
Fitting 5 folds for each of 6 candidates, totalling 30 fits
Best params (n_neighbors): {'kneighborsclassifier__n_neighbors': 3}
(5-fold CV aggregated predictions) Balanced Accuracy: 0.4509527909652719
              precision    recall  f1-score   support

        High     0.4065    0.3183    0.3570       355
         Low     0.3408    0.2000    0.2521       455
      Medium     0.7266    0.8345    0.7768      1783

    accuracy                         0.6525      2593
   macro avg     0.4913    0.4510    0.4620      2593
weighted avg     0.6151    0.6525    0.6273      2593
```



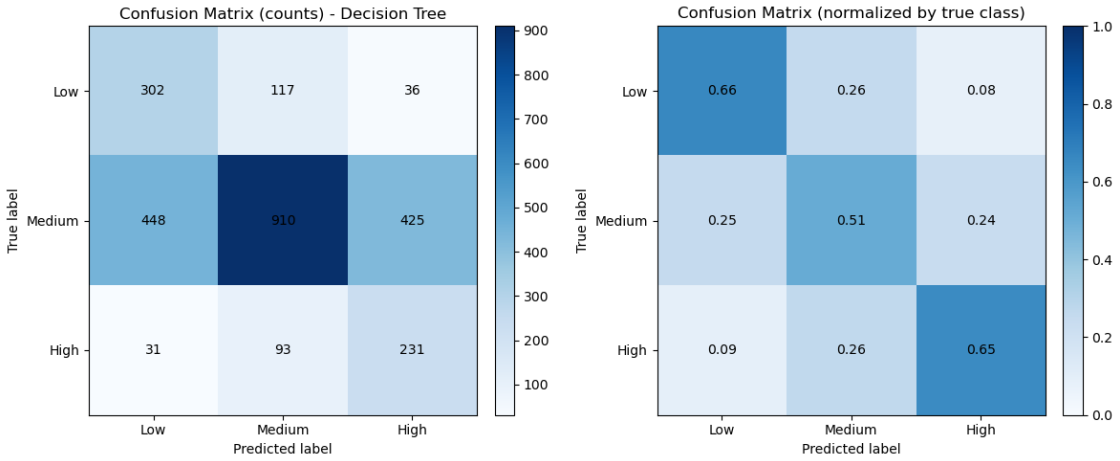
6 Tried with decision tree and random forest model using balanced weights

6.1 Doing hyperparameter tuning on max_depth,criteria,minimumsmaples to split and #treess(only for random forest)

[9]:
```
# ensure labels align with features
y = y_transform.loc[X_no_outliers.index]

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# base classifiers
classifiers = {
    "Decision Tree": DecisionTreeClassifier(random_state=42,␣
 ↪class_weight='balanced'),
    "Random Forest": RandomForestClassifier(random_state=42, n_jobs=-1,␣
 ↪class_weight='balanced')
}

# param grids for grid search
param_grids = {
    "Decision Tree": {
        "max_depth": [3, 5, 10, None],
        "min_samples_split": [2, 5, 10],
        "criterion": ["gini", "entropy"]
    },
    "Random Forest": {
        "n_estimators": [100, 200],
        "max_depth": [3,5, 10, None],
        "min_samples_split": [2, 5, 10],
        "criterion": ["gini", "entropy"]
    }
}

for name, clf in classifiers.items():
    print(f"\n=== {name} - GridSearchCV ===")
    grid = GridSearchCV(clf, param_grids[name], cv=cv,␣
 ↪scoring='balanced_accuracy', n_jobs=-1, verbose=1)
    grid.fit(X_no_outliers, y)
    print("Best params:", grid.best_params_)
    print("Best CV balanced_accuracy:", grid.best_score_)

    best = grid.best_estimator_
    # get cross-validated predictions using the best estimator (unbiased CV)
```

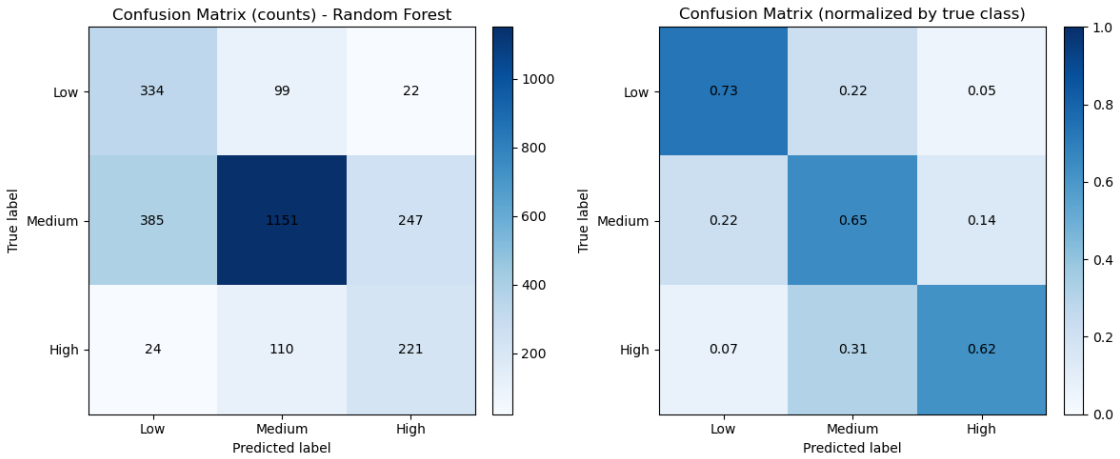
```
    y_pred = cross_val_predict(best, X_no_outliers, y, cv=cv, n_jobs=-1)
    print(f"\n{name} (5-fold CV aggregated predictions) Balanced Accuracy:",␣
↪balanced_accuracy_score(y, y_pred))
    print(classification_report(y, y_pred, digits=4))
    plot_confusion_matrix(y, y_pred)
```

```
=== Decision Tree - GridSearchCV ===
Fitting 5 folds for each of 24 candidates, totalling 120 fits
Best params: {'criterion': 'gini', 'max_depth': 5, 'min_samples_split': 2}
Best CV balanced_accuracy: 0.608265064255867

Decision Tree (5-fold CV aggregated predictions) Balanced Accuracy:
0.6082720867535193
              precision    recall  f1-score   support

        High     0.3338    0.6507    0.4413       355
         Low     0.3867    0.6637    0.4887       455
      Medium     0.8125    0.5104    0.6269      1783

    accuracy                         0.5565      2593
   macro avg     0.5110    0.6083    0.5190      2593
weighted avg     0.6722    0.5565    0.5773      2593
```




```
=== Random Forest - GridSearchCV ===
Fitting 5 folds for each of 48 candidates, totalling 240 fits
Best params: {'criterion': 'entropy', 'max_depth': 10, 'min_samples_split': 10,
'n_estimators': 100}
Best CV balanced_accuracy: 0.6673820349634655
```

```
Random Forest (5-fold CV aggregated predictions) Balanced Accuracy:
0.6673807893306601
              precision    recall  f1-score   support

        High     0.4510    0.6225    0.5231       355
         Low     0.4495    0.7341    0.5576       455
      Medium     0.8463    0.6455    0.7324      1783

    accuracy                         0.6579      2593
   macro avg     0.5823    0.6674    0.6044      2593
weighted avg     0.7226    0.6579    0.6731      2593
```



6.2 Neural Net doesn't seem to improve a lot

[10]:
```python
import numpy as np
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import classification_report, accuracy_score
import tensorflow as tf
from tensorflow.keras import models, layers, callbacks

# Neural network classifier (Keras) for the existing 3-class target (Low/Medium/
  ↪High)

# Prepare features and labels (use existing X_no_outliers and y)
X_nn = X_no_outliers.copy().astype(float)            # features (2593 x n)
le = LabelEncoder()
y_enc = le.fit_transform(y.loc[X_nn.index])          # encode strings -> 0/1/2
y_cat = tf.keras.utils.to_categorical(y_enc)         # one-hot for training
```

```
# Train/test split (stratified)
X_train, X_test, y_train, y_test, y_train_enc, y_test_enc = train_test_split(
    X_nn, y_cat, y_enc, test_size=0.2, stratify=y_enc, random_state=42
)

# Scale features (fit on train only)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Compute class weights to handle imbalance
classes = np.unique(y_train_enc)
cw_vals = compute_class_weight(class_weight='balanced', classes=classes,␣
 ↪y=y_train_enc)
class_weight = {int(c): float(w) for c, w in zip(classes, cw_vals)}

# Build a small feed-forward network
input_dim = X_train_scaled.shape[1]
model = models.Sequential([
    layers.Input(shape=(input_dim,)),
    layers.Dense(128, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.3),
    layers.Dense(64, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.2),
    layers.Dense(3, activation='softmax')
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Callbacks
es = callbacks.EarlyStopping(monitor='val_loss', patience=12,␣
 ↪restore_best_weights=True)
rlr = callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=6,␣
 ↪verbose=1)

# Train
history = model.fit(
    X_train_scaled, y_train,
    validation_split=0.1,
    epochs=100,
```

```
    batch_size=32,
    class_weight=class_weight,
    callbacks=[es, rlr],
    verbose=2
)

# Evaluate on test set
y_proba = model.predict(X_test_scaled)
y_pred_idx = np.argmax(y_proba, axis=1)
y_test_idx = np.argmax(y_test, axis=1)

y_pred_labels = le.inverse_transform(y_pred_idx)
y_true_labels = le.inverse_transform(y_test_idx)

print("Test accuracy:", balanced_accuracy_score(y_true_labels, y_pred_labels))
print(classification_report(y_true_labels, y_pred_labels, digits=4))

# Reuse existing confusion matrix plotting helper
plot_confusion_matrix(pd.Series(y_true_labels, name='y_true'), pd.
  ↪Series(y_pred_labels, name='y_pred'))

# Plot training curves
import matplotlib.pyplot as plt
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.plot(history.history['loss'], label='train loss')
plt.plot(history.history['val_loss'], label='val loss')
plt.legend(); plt.title('Loss')
plt.subplot(1,2,2)
plt.plot(history.history['accuracy'], label='train acc')
plt.plot(history.history['val_accuracy'], label='val acc')
plt.legend(); plt.title('Accuracy')
plt.tight_layout(); plt.show()
```

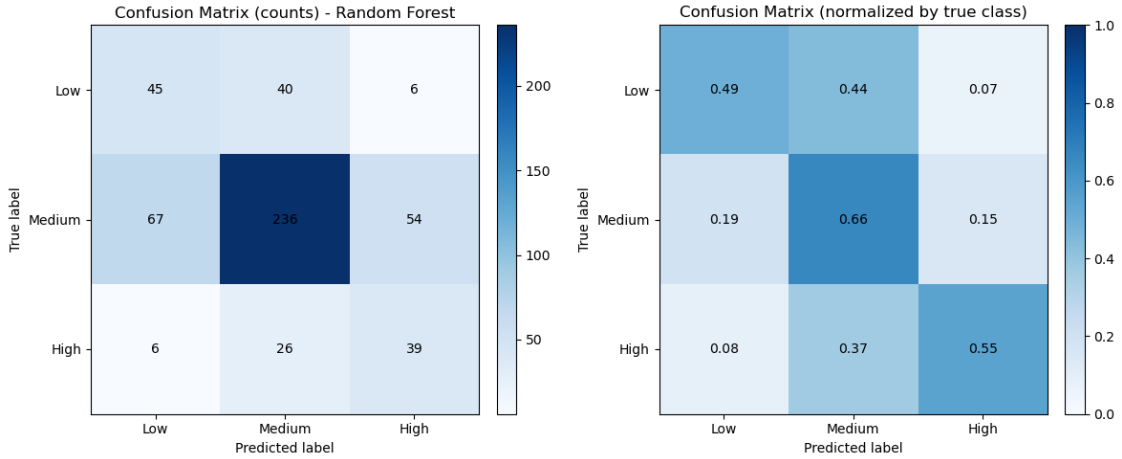
```
Epoch 1/100
59/59 - 1s - 25ms/step - accuracy: 0.3778 - loss: 1.4160 - val_accuracy: 0.3606
- val_loss: 1.1713 - learning_rate: 1.0000e-03
Epoch 2/100
59/59 - 0s - 1ms/step - accuracy: 0.4427 - loss: 1.0576 - val_accuracy: 0.4327 -
val_loss: 1.0928 - learning_rate: 1.0000e-03
Epoch 3/100
59/59 - 0s - 1ms/step - accuracy: 0.4861 - loss: 0.9636 - val_accuracy: 0.4615 -
val_loss: 1.0400 - learning_rate: 1.0000e-03
Epoch 4/100
59/59 - 0s - 1ms/step - accuracy: 0.5263 - loss: 0.8559 - val_accuracy: 0.4375 -
val_loss: 1.0414 - learning_rate: 1.0000e-03
Epoch 5/100
59/59 - 0s - 1ms/step - accuracy: 0.5354 - loss: 0.8168 - val_accuracy: 0.4952 -
```

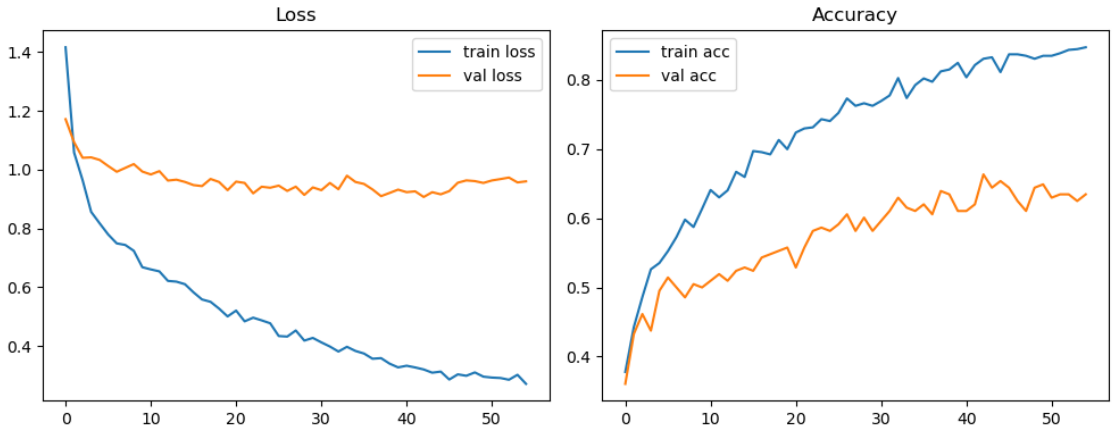
```
val_loss: 1.0327 - learning_rate: 1.0000e-03
Epoch 6/100
59/59 - 0s - 1ms/step - accuracy: 0.5525 - loss: 0.7789 - val_accuracy: 0.5144 -
val_loss: 1.0120 - learning_rate: 1.0000e-03
Epoch 7/100
59/59 - 0s - 1ms/step - accuracy: 0.5729 - loss: 0.7487 - val_accuracy: 0.5000 -
val_loss: 0.9924 - learning_rate: 1.0000e-03
Epoch 8/100
59/59 - 0s - 1ms/step - accuracy: 0.5981 - loss: 0.7434 - val_accuracy: 0.4856 -
val_loss: 1.0056 - learning_rate: 1.0000e-03
Epoch 9/100
59/59 - 0s - 1ms/step - accuracy: 0.5874 - loss: 0.7235 - val_accuracy: 0.5048 -
val_loss: 1.0186 - learning_rate: 1.0000e-03
Epoch 10/100
59/59 - 0s - 1ms/step - accuracy: 0.6136 - loss: 0.6677 - val_accuracy: 0.5000 -
val_loss: 0.9931 - learning_rate: 1.0000e-03
Epoch 11/100
59/59 - 0s - 1ms/step - accuracy: 0.6409 - loss: 0.6601 - val_accuracy: 0.5096 -
val_loss: 0.9833 - learning_rate: 1.0000e-03
Epoch 12/100
59/59 - 0s - 1ms/step - accuracy: 0.6302 - loss: 0.6535 - val_accuracy: 0.5192 -
val_loss: 0.9946 - learning_rate: 1.0000e-03
Epoch 13/100
59/59 - 0s - 1ms/step - accuracy: 0.6404 - loss: 0.6211 - val_accuracy: 0.5096 -
val_loss: 0.9624 - learning_rate: 1.0000e-03
Epoch 14/100
59/59 - 0s - 1ms/step - accuracy: 0.6672 - loss: 0.6186 - val_accuracy: 0.5240 -
val_loss: 0.9655 - learning_rate: 1.0000e-03
Epoch 15/100
59/59 - 0s - 1ms/step - accuracy: 0.6597 - loss: 0.6098 - val_accuracy: 0.5288 -
val_loss: 0.9578 - learning_rate: 1.0000e-03
Epoch 16/100
59/59 - 0s - 1ms/step - accuracy: 0.6972 - loss: 0.5822 - val_accuracy: 0.5240 -
val_loss: 0.9471 - learning_rate: 1.0000e-03
Epoch 17/100
59/59 - 0s - 1ms/step - accuracy: 0.6956 - loss: 0.5579 - val_accuracy: 0.5433 -
val_loss: 0.9436 - learning_rate: 1.0000e-03
Epoch 18/100
59/59 - 0s - 1ms/step - accuracy: 0.6924 - loss: 0.5499 - val_accuracy: 0.5481 -
val_loss: 0.9679 - learning_rate: 1.0000e-03
Epoch 19/100
59/59 - 0s - 1ms/step - accuracy: 0.7133 - loss: 0.5268 - val_accuracy: 0.5529 -
val_loss: 0.9577 - learning_rate: 1.0000e-03
Epoch 20/100
59/59 - 0s - 2ms/step - accuracy: 0.6999 - loss: 0.5001 - val_accuracy: 0.5577 -
val_loss: 0.9298 - learning_rate: 1.0000e-03
Epoch 21/100
59/59 - 0s - 1ms/step - accuracy: 0.7240 - loss: 0.5201 - val_accuracy: 0.5288 -
```

```
val_loss: 0.9589 - learning_rate: 1.0000e-03
Epoch 22/100
59/59 - 0s - 1ms/step - accuracy: 0.7299 - loss: 0.4834 - val_accuracy: 0.5577 -
val_loss: 0.9544 - learning_rate: 1.0000e-03
Epoch 23/100
59/59 - 0s - 1ms/step - accuracy: 0.7315 - loss: 0.4959 - val_accuracy: 0.5817 -
val_loss: 0.9190 - learning_rate: 1.0000e-03
Epoch 24/100
59/59 - 0s - 1ms/step - accuracy: 0.7433 - loss: 0.4867 - val_accuracy: 0.5865 -
val_loss: 0.9415 - learning_rate: 1.0000e-03
Epoch 25/100
59/59 - 0s - 1ms/step - accuracy: 0.7406 - loss: 0.4768 - val_accuracy: 0.5817 -
val_loss: 0.9381 - learning_rate: 1.0000e-03
Epoch 26/100
59/59 - 0s - 1ms/step - accuracy: 0.7524 - loss: 0.4334 - val_accuracy: 0.5913 -
val_loss: 0.9452 - learning_rate: 1.0000e-03
Epoch 27/100
59/59 - 0s - 1ms/step - accuracy: 0.7733 - loss: 0.4316 - val_accuracy: 0.6058 -
val_loss: 0.9273 - learning_rate: 1.0000e-03
Epoch 28/100
59/59 - 0s - 1ms/step - accuracy: 0.7626 - loss: 0.4524 - val_accuracy: 0.5817 -
val_loss: 0.9417 - learning_rate: 1.0000e-03
Epoch 29/100
59/59 - 0s - 1ms/step - accuracy: 0.7663 - loss: 0.4179 - val_accuracy: 0.6010 -
val_loss: 0.9136 - learning_rate: 1.0000e-03
Epoch 30/100
59/59 - 0s - 1ms/step - accuracy: 0.7626 - loss: 0.4271 - val_accuracy: 0.5817 -
val_loss: 0.9392 - learning_rate: 1.0000e-03
Epoch 31/100
59/59 - 0s - 1ms/step - accuracy: 0.7696 - loss: 0.4118 - val_accuracy: 0.5962 -
val_loss: 0.9298 - learning_rate: 1.0000e-03
Epoch 32/100
59/59 - 0s - 1ms/step - accuracy: 0.7776 - loss: 0.3977 - val_accuracy: 0.6106 -
val_loss: 0.9543 - learning_rate: 1.0000e-03
Epoch 33/100
59/59 - 0s - 1ms/step - accuracy: 0.8028 - loss: 0.3807 - val_accuracy: 0.6298 -
val_loss: 0.9330 - learning_rate: 1.0000e-03
Epoch 34/100
59/59 - 0s - 1ms/step - accuracy: 0.7738 - loss: 0.3968 - val_accuracy: 0.6154 -
val_loss: 0.9790 - learning_rate: 1.0000e-03
Epoch 35/100

Epoch 35: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
59/59 - 0s - 1ms/step - accuracy: 0.7926 - loss: 0.3826 - val_accuracy: 0.6106 -
val_loss: 0.9577 - learning_rate: 1.0000e-03
Epoch 36/100
59/59 - 0s - 1ms/step - accuracy: 0.8023 - loss: 0.3739 - val_accuracy: 0.6202 -
val_loss: 0.9510 - learning_rate: 5.0000e-04
```

```
Epoch 37/100
59/59 - 0s - 1ms/step - accuracy: 0.7974 - loss: 0.3562 - val_accuracy: 0.6058 -
val_loss: 0.9322 - learning_rate: 5.0000e-04
Epoch 38/100
59/59 - 0s - 1ms/step - accuracy: 0.8124 - loss: 0.3582 - val_accuracy: 0.6394 -
val_loss: 0.9094 - learning_rate: 5.0000e-04
Epoch 39/100
59/59 - 0s - 1ms/step - accuracy: 0.8151 - loss: 0.3395 - val_accuracy: 0.6346 -
val_loss: 0.9203 - learning_rate: 5.0000e-04
Epoch 40/100
59/59 - 0s - 1ms/step - accuracy: 0.8248 - loss: 0.3267 - val_accuracy: 0.6106 -
val_loss: 0.9317 - learning_rate: 5.0000e-04
Epoch 41/100
59/59 - 0s - 1ms/step - accuracy: 0.8039 - loss: 0.3324 - val_accuracy: 0.6106 -
val_loss: 0.9230 - learning_rate: 5.0000e-04
Epoch 42/100
59/59 - 0s - 1ms/step - accuracy: 0.8215 - loss: 0.3265 - val_accuracy: 0.6202 -
val_loss: 0.9259 - learning_rate: 5.0000e-04
Epoch 43/100
59/59 - 0s - 1ms/step - accuracy: 0.8307 - loss: 0.3197 - val_accuracy: 0.6635 -
val_loss: 0.9066 - learning_rate: 5.0000e-04
Epoch 44/100
59/59 - 0s - 1ms/step - accuracy: 0.8328 - loss: 0.3087 - val_accuracy: 0.6442 -
val_loss: 0.9229 - learning_rate: 5.0000e-04
Epoch 45/100
59/59 - 0s - 1ms/step - accuracy: 0.8114 - loss: 0.3125 - val_accuracy: 0.6538 -
val_loss: 0.9156 - learning_rate: 5.0000e-04
Epoch 46/100
59/59 - 0s - 1ms/step - accuracy: 0.8371 - loss: 0.2854 - val_accuracy: 0.6442 -
val_loss: 0.9267 - learning_rate: 5.0000e-04
Epoch 47/100
59/59 - 0s - 1ms/step - accuracy: 0.8371 - loss: 0.3031 - val_accuracy: 0.6250 -
val_loss: 0.9551 - learning_rate: 5.0000e-04
Epoch 48/100
59/59 - 0s - 1ms/step - accuracy: 0.8349 - loss: 0.2982 - val_accuracy: 0.6106 -
val_loss: 0.9630 - learning_rate: 5.0000e-04
Epoch 49/100

Epoch 49: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
59/59 - 0s - 1ms/step - accuracy: 0.8307 - loss: 0.3097 - val_accuracy: 0.6442 -
val_loss: 0.9607 - learning_rate: 5.0000e-04
Epoch 50/100
59/59 - 0s - 1ms/step - accuracy: 0.8349 - loss: 0.2952 - val_accuracy: 0.6490 -
val_loss: 0.9544 - learning_rate: 2.5000e-04
Epoch 51/100
59/59 - 0s - 1ms/step - accuracy: 0.8349 - loss: 0.2921 - val_accuracy: 0.6298 -
val_loss: 0.9628 - learning_rate: 2.5000e-04
Epoch 52/100
```

```
59/59 - 0s - 1ms/step - accuracy: 0.8387 - loss: 0.2906 - val_accuracy: 0.6346 -
val_loss: 0.9675 - learning_rate: 2.5000e-04
Epoch 53/100
59/59 - 0s - 1ms/step - accuracy: 0.8435 - loss: 0.2847 - val_accuracy: 0.6346 -
val_loss: 0.9727 - learning_rate: 2.5000e-04
Epoch 54/100
59/59 - 0s - 1ms/step - accuracy: 0.8446 - loss: 0.3016 - val_accuracy: 0.6250 -
val_loss: 0.9561 - learning_rate: 2.5000e-04
Epoch 55/100

Epoch 55: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.
59/59 - 0s - 1ms/step - accuracy: 0.8473 - loss: 0.2707 - val_accuracy: 0.6346 -
val_loss: 0.9598 - learning_rate: 2.5000e-04
17/17             0s 3ms/step
Test accuracy: 0.5682885649745634
              precision    recall  f1-score   support

        High     0.3939    0.5493    0.4588        71
         Low     0.3814    0.4945    0.4306        91
      Medium     0.7815    0.6611    0.7162       357

    accuracy                         0.6166       519
   macro avg     0.5189    0.5683    0.5352       519
weighted avg     0.6583    0.6166    0.6309       519
```




Loss
train loss
val loss
1.4
1.2
1.0
0.8
0.6
0.4
0
10
20
30
40
50
Accuracy
train acc
val acc
0.8
0.7
0.6
0.5
0.4
0
10
20
30
40
50